エネルギー工学 I 課題 2

ME2208 髙橋 尚太郎

2023 年 2 月 20 日

明石工業高等専門学校 専攻科 機械・電子システム工学専攻 1 年

発表概要

1. 取り組んだ課題
2. 目標
3. モデル
4. 計算原理
5. 実装
6. 実験結果
7. 考察・結論
8. おわりに

取り組んだ課題

■ 対流型サウナ室内の熱気の移動に関するシミュレーション

対流型サウナの概要: サウナストーンを積んだサウナ　　対流の仕組み：空気の膨張（上昇）、収縮（下降）。

[参考] 知っているようで知らないサウナの「熱」の正体：アットダイム https://dime.jp/genre/1454460/

どのように熱が伝わるかが知りたい

目標

- サウナ室が**なるべく早く温まる条件**を求める
- サウナ室が**なるべく快適な条件**を見つける

方法

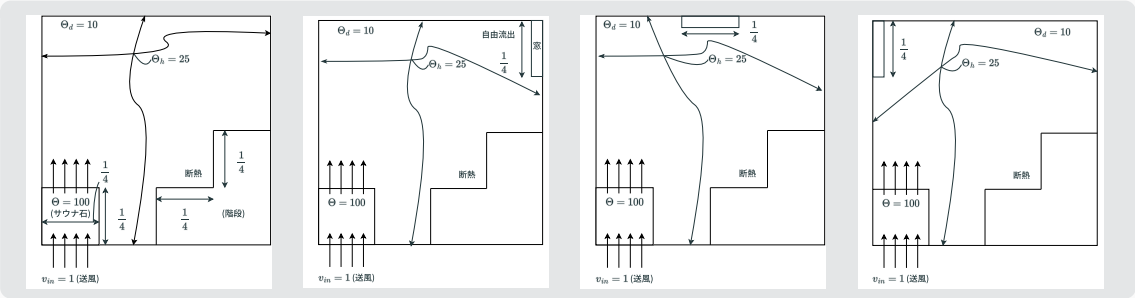
サウナ室全体が温まるまでの所要時間を求める
窓を付けた場合の温まり方のばらつきを調べる

ex.1 所要時間の場合
→ サウナ室全体が**均一に温まった状態**での実行サイクル数を比較する

ex.2 ばらつきの場合
→ **同一実行サイクル数**でサウナ室の温度分布を比較する

対流型サウナの部屋のモデルを作成する

モデル



境界条件の設定

サウナ石 $P = 0, \Theta = 100$

(窓) 自由流出 $\partial P/\partial X = \partial U/\partial X = \partial \Theta_h/\partial X = 0, V = 0$ (左右面の場合)

断熱条件 $\partial \Theta/\partial X = 0, (\Theta/\partial Y = 0)$

計算原理

所要時間 t を求める式

$$Re = u_0 x_0 / \nu, (\nu\text{:動粘性係数(物性値)}) \tag{1}$$

$$t_0 = x_0 / u_0 \tag{2}$$

$$t = t_0 T \tag{3}$$

既知の値

Re：パラメータファイルで指定, x_0：1 としている,
T：時間の刻み幅 (DT)× ステップ数 (NCYCLE)

流体の物性値が分かれば
レイノルズ数と無次元化の関係式から所要時間を求めることができる!!

実装

- 実行環境：Visual Studio Code
- コンパイラ：MinGW-w64 GCC (Minimalist GNU for Windows) クロスコンパイラ
- OS：Windows 10 Pro, メモリ：8GB, SSD：256GB

cinit 関数

```
for (ix = 0; ix <= NX + 1; ix++)
{ /*TDの初期値設定(境界内は乗温(+0.5)と仮定)*/
  for (iy = 0; iy <= NY + 1; iy++)
  {
    TN[ix][iy] = 10.0;  /*いきなり平均温度
  }                     /*注意！浮動小数
}


for (iy = 1; iy <= NY + 1; iy++)
{ /*TDの境界：左側壁*/
  TN[1][iy] = TN[0][iy];
}
for (ix = 1; ix <= NX; ix++)
{ /*TDの境界：上面（断熱）*/
  TN[ix][NY + 1] = TN[ix][NY];
}
for (ix = 1; ix <= NX; ix++)
{ /*TDの境界：下面（断熱）*/
  TN[ix][1] = TN[ix][0];
}
for (iy=0;iy<=NY+1;iy++)
{/*TDの境界：右側面（断熱）*/
  TN[NX+1][iy] = TN[NX][iy];
}
```

階段部の速度の境界条件

```
//右端
for (iy = 1; iy <= NY*1/4; iy++){
    UN[NX*1/4][iy] = 0;
    VN[NX*1/4][iy] = -VN[(NX*1/4)+1][iy];
}
//真ん中端
for (iy = 1; iy <= NY * 1/4; iy++){
    UN[NX*2/4][iy] = 0;
    VN[NX*2/4][iy] = -VN[(NX*2/4)+1][iy];
}
//真ん中から右にかけて
for (ix = NX*2/4; ix <= NX*3/4; ix++){
    UN[ix][NY*1/4] = -UN[ix][(NY*1/4)+1];
    VN[ix][NY*1/4] = 0;
}
//右から上
for (iy = NY*1/4; iy <= NY*2/4 ; iy++){
    UN[NX*3/4][iy] = 0;
    VN[NX*3/4][iy] = -VN[(NX*3/4)+1][iy];
}
//右上から右
for (ix = NX*3/4; ix <= NX; ix++){
    UN[ix][NY*2/4] = -UN[ix][(NY*2/4)+1];
    VN[ix][NY*2/4] = 0;
}
```

断熱条件

```
for (iy = 0; iy <= NY + 1; iy++)
{ /*右側壁*/
    TN[NX + 1][iy] = 2.0 * (+25) - TN[NX][iy];
}
//真ん中端
for (iy = 1; iy <= NY * 1/4; iy++){
    TN[NX*2/4][iy] = TN[(NX*2/4)+1][iy];
}
//真ん中から右にかけて
for (ix = NX*2/4; ix <= NX*3/4; ix++){
    TN[ix][NY*1/4] = TN[ix][(NY*1/4)+1];
}
//右から上
for (iy = NY*1/4; iy <= NY*2/4 ; iy++){
    TN[NX*3/4][iy] = TN[(NX*3/4)+1][iy];
}


for (iy = 0; iy <= NY + 1; iy++)
{ /*左側壁*/
    TN[0][iy] = 2.0 * (+25) - TN[1][iy];
}
//溝
for (ix = 1; ix <= NX*1/4; ix++){
    TN[ix][(NY*1/4)-1] = TN[ix][NY*1/4];
}
//溝
for (iy = 1; iy < NY*1/4; iy++){
    TN[NX*1/4][iy] = TN[(NX*1/4)+1][iy];
}
```

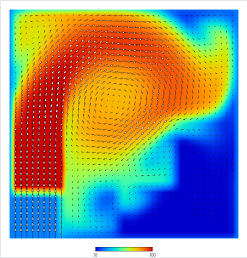
熱流の吹き出し口

```
for (ix = 1; ix <= NX; ix++)
{ /*上面*/
  TN[ix][NY + 1] = 2*(+25)-TN[ix+1][NY+1];
  //TN[ix][NY + 1] = TN[ix][NY];
}
for (ix = 1; ix <= NX; ix++)
{ /*下面*/
    TN[ix][0] = 2*(+25)-TN[ix+1][0];
}
for (ix = 1; ix <= NX; ix++){
    if (ix < NX*1/4){
      TN[ix][NY*1/4] = 2.0 * (+100) - TN[ix+1][NY*1/4];
    }
}
```

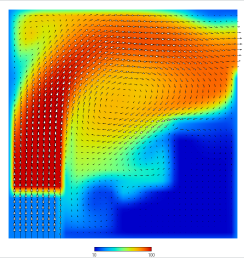
実験結果 1/3 (動画)

実験結果 2/3

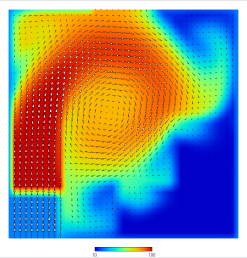
■ 所要時間に関して（目標との照合 1/2）



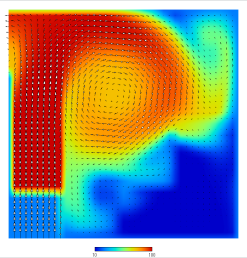
窓無し NCYCLE=55000, (3.4 時間)



右壁面に窓 NCYCLE=45000, **(2.8 時間)**



上面に窓 NCYCLE=50000, (3.1 時間)

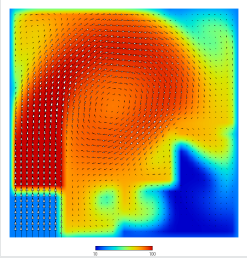


左壁面に窓 NCYCLE=65000, (3.9 時間)

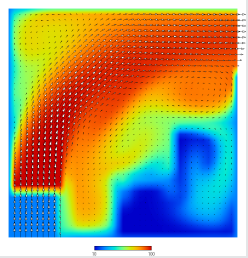
右壁面に窓があると部屋が早く温まる!!
その後 NCYCLE=100000(6.1 時間) までシミュレーションを実行

実験結果 3/3

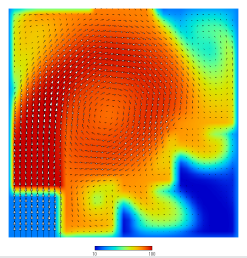
■ 温度分布のばらつきについて（目標との照合 2/2）



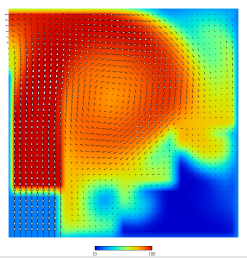
窓無し 基準



右壁面に窓（**熱気が上に逃げる**）



上面に窓（**均等に温度が分布**）



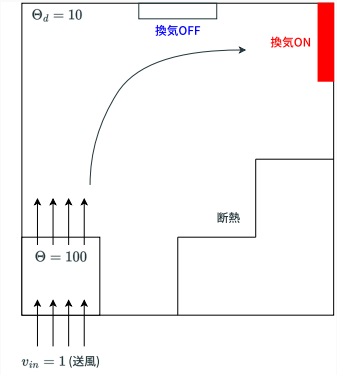
ばらつきあり

上面に窓を付けても熱気の流れに変化がなく温度が均等に分布
右壁面に窓がある場合温まりやすく冷めやすい!!

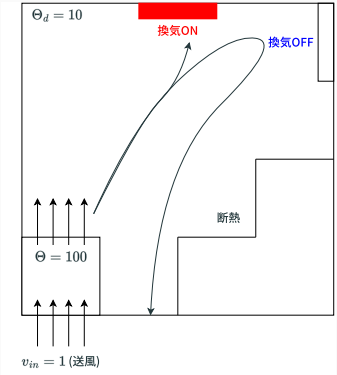
考察・結論

考察 早く温まる場合・温度分布のばらつきが少ない場合が存在 → **窓の取り付け位置を組み合わせる**
結論 窓を複数個所に設置して時間ごとに換気のタイミングを制御する仕組み

部屋全体が温まるまでは右側窓を換気 (上面窓は OFF)



部屋全体が温まると上面窓を換気 (右側窓は OFF)



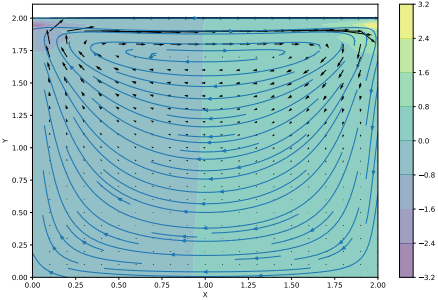
おわりに

- サウナ室が**なるべく早く温まる条件** → 右壁面に窓を付ける
- サウナ室が**なるべく快適な条件** → 上面の窓と右壁面の窓の換気制御

- 改善点

python を使って分かりやすく

ex.1 ファイルの出力 → グラフ化 → 画像出力
→ 動画出力 → これらの処理を自動で行いたい

ex.2 Python は可視化ツールが豊富で数値解析に便利

[参考] 2 次元キャビティ流れを Python で実装する `https://takun-physics.net/10217/`